

Project: Gourmet-Go Distributed Order System (Full Stack + DevOps)

A. Project Objective

Transform the initial gRPC orchestration lab (Lab5) into a complete production-like distributed system with:

- Microservices architecture
- Real databases (one per service)
- UI for order creation
- Dockerized deployment
- CI/CD pipeline

The system must implement a **Saga orchestration pattern with compensation**.

B. Target Architecture

You will implement the following services:

1. **Order-Orchestrator**
 - Central coordinator (Saga orchestrator)
 - Calls all other services using gRPC
2. **Order-Service**
 - Stores order state (APPROVAL_PENDING, APPROVED, REJECTED)
3. **Kitchen-Service**
 - Manages ticket creation
4. **Accounting-Service**
 - Handles payment authorization

C. Workflow (Business Logic)

1. Happy Path

1. User creates an order via UI
2. Orchestrator:
 - Update order → **APPROVAL_PENDING**
 - Create kitchen ticket
 - Authorize payment (how payment is authorized is your choice)
 - If all succeed → update order → **APPROVED**

2. Compensation Path (Saga Rollback)

If payment fails:

1. Reject kitchen ticket
2. Update order → REJECTED

D. Functional Requirements

1. Front-End Application

Students must implement a simple UI (choice of tech: React, Angular, or simple HTML/JS):

Features:

- Create a new order (orderId, amount)
- View order status
- Display success/failure messages

2. gRPC Communication

All inter-service communication must use **gRPC**

3. Database Integration (MANDATORY)

Each microservice must have its own database.

Constraints:

- Use MySQL or PostgreSQL
- Each DB runs in a separate Docker container

4. Dockerization

Each component must have:

- A **Dockerfile**
- A working container

You must also provide:

- `docker-compose.yml` including:
 - All microservices
 - Databases
 - Network configuration

5. CI/CD Pipeline (MANDATORY)

Students must implement a pipeline using:

- GitHub Actions or GitLab CI/CD

Pipeline stages:

1. Build (Maven)
2. Package (JAR)
3. Build Docker images
4. Push images to Docker Hub

E. Allowed Extensions

Students are encouraged to extend and improve the system beyond the mandatory scope.

Examples of possible extensions:

- Add new features:
 - Order cancellation from UI
 - Order tracking dashboard
 - Display detailed order history
 - Retry mechanism for failed operations
 - ...
- Improve architecture:
 - Add an API Gateway
 - Introduce REST endpoints for UI access
- Improve DevOps:
 - Add monitoring (Prometheus/Grafana)

F. Project Evaluation and Assessment

The project will be evaluated based on both functional correctness and DevOps implementation quality. As part of the assessment, the instructor will verify the following elements:

1. CI/CD Pipeline Validation

The CI/CD pipeline implemented using GitHub Actions must be fully operational.

The pipeline should successfully:

- Build the project
- Run tests (if implemented)
- Generate executable artifacts
- Build Docker images

2. Docker Image Publication

All microservice images must be correctly pushed to Docker Hub.

Image naming and versioning should be clear and consistent.

3. Deployment with Docker Compose

The instructor will attempt to run the entire system using: `docker-compose up`

Images must be **pulled from Docker Hub** (not built locally during evaluation).

The system must start successfully with all services and databases correctly connected.

4. Functional Validation

Normal Execution (Happy Path)

- The instructor will test a complete successful workflow:
 - Order creation via UI
 - Execution of all service calls
 - Final status = **APPROVED**

Compensation Execution (Failure Path)

- The instructor will simulate a failure scenario (e.g., payment rejection):
 - Verification that compensation logic is triggered
 - Kitchen ticket is cancelled
 - Final status = **REJECTED**